

Reviewer Pack — Minimal Local Dimension of Coxeter Groups Bounds Resolution ...

Entry 7890ffec5d38 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 11:27:24 UTC

Minimal Local Dimension of Coxeter Groups Bounds Resolution Proof Width

Entry ID: 7890ffec5d38 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 11:27:24 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Coxeter Groups) **Field B** (complexity object): Boolean Function Complexity: Resolution Proof Complexity

Statement:

For every satisfiable Boolean formula φ with n variables, the local dimension of its associated Coxeter group $\Gamma(\varphi)$ is upper-bounded by a function $f(n)$, and this bound determines the resolution proof width for φ .

Rationale (proposer's reasoning):

The study of Coxeter groups in combinatorial geometry has shown that certain properties can be related to algebraic structures. If such a relationship exists with resolution proof complexity, it could provide new insights into the inherent structure of Boolean formulas.

Taxonomy category: CG_KW_ANDREEV (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 68c1b798dc3811e7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the local dimension of Coxeter group $\Gamma(\varphi)$ for a Boolean formula φ with n variables exceeds $f(n)$, or if there is no significant correlation between the local dimension and resolution proof width (p-value > 0.05), then the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter groups" AND "resolution proof complexity" - "Boolean functions" AND "local dimension of Coxeter groups" - "resolution proof width" AND "minimal local dimension Coxeter groups"

Top relevant hits considered: - [s2:10.1109/TNS.2025.3602984] A Compton Camera Resolution Enhancement by Increasing the Number of Sensors per Readout Channel - [s2:1602.04412] Reductive groups, the loop Grassmannian, and the Springer resolution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_formula(n: int) -> str:
    if n == 1:
        return "True"
    elif n == 2:
        return "A & B"
    else:
        formulas = [generate_formula(i) for i in range(1, n)]
        return f"({' & '.join(formulas[:n//2])}) | ({' & '.join(formulas[n//2:])})"

def local_dimension(formula: str) -> int:
    if formula == "True":
        return 0
    elif formula == "False":
        return float('inf')
    elif ' & ' in formula:
        left, right = formula.split(' & ')
        return max(local_dimension(left), local_dimension(right))
    elif ' | ' in formula:
        left, right = formula.split(' | ')
        return 1 + min(local_dimension(left), local_dimension(right))

def resolution_width(formula: str) -> int:
    if formula == "True":
        return 0
    elif formula == "False":
        return float('inf')
    elif ' & ' in formula:
```

```

        left, right = formula.split(' & ')
        return max(resolution_width(left), resolution_width(right))
    elif '|' in formula:
        left, right = formula.split(' | ')
        return 1 + min(resolution_width(left), resolution_width(right))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        formula = generate_formula(n)
        ld = local_dimension(formula)
        rw = resolution_width(formula)

        if ld > rw:
            return {
                "metric_name": "local_dimension",
                "metric_value": ld,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": f"Formula '{formula}' has local dimension {ld} > resolution width {rw}"
            }

        results.append((ld, rw))

    avg_ld = sum(ld for ld, _ in results) / len(results)
    avg_rw = sum(rw for _, rw in results) / len(results)

    return {
        "metric_name": "local_dimension",
        "metric_value": avg_ld,
        "instances_tested": len(results),
        "n_max": 40,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    avg_ld = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={avg_ld} std=0.0 support_fraction=1.0")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={avg_ld} std=0.0 support_fraction={support_fraction:.2f}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con"]
    print(f"RESULT: FALSIFIED counterexample='local_dimension > resolution_width' first_failing")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4825ad26.py", line 93, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4825ad26.py", line 57, in run_trial
    ld = local_dimension(formula)
          ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4825ad26.py", line 33, in local_dimension
    left, right = formula.split(' & ')
    ^^^^^^^^^
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that neither the support nor falsification conditions could be checked. | next: Re-run the test to ensure it completes successfully and produces the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21475
2	propose	ollama_remote	glm4:latest	0	0	24325
3	propose	ollama_remote	glm4:latest	0	0	16272
4	propose	ollama_remote	glm4:latest	0	0	15403
5	preregistration	ollama_remote	glm4:latest	0	0	9136
6	novelty	ollama_remote	glm4:latest	0	0	8171
7	novelty	ollama_remote	glm4:latest	0	0	9047
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12735

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8688
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9080
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25502
12	judge	ollama_remote	glm4:latest	0	0	10801

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 170635 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in research/audit/7890ffec5d38.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7890ffec5d38.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7890ffec5d38.tar.gz (if generated)